

- 1) 硬件安全需求和相关安全机制的属性来控制硬件单元的内部失效,这包括内部安全机制覆盖瞬态故障,例如,使用的技术。相关属性可以包括定时器和看门狗检测
- 2) 硬件安全需求和相关安全机制的属性能够承受外部单元的失效。例如在 ECU 外部 失效时,对 ECU 输入开路
- 3) 硬件安全需求和相关安全机制的属性能够匹配别的单元的安全需求
- 4) 硬件安全需求和相关安全机制的属性能够检测和指示内部和外部故障
- 5) 硬件安全需求不指定安全机制 产品硬件的设计验证标准,包括环境条件(温度,振动,电磁干扰,等),具体的操作环境(电源电压,任务历程等)和特定组件的要求。

硬件安全要求应按照 ISO 26262-8:2011 条款 6 和 9 验证,具有以下属性:

- 1) 与技术安全概念、系统设计规范、硬件设计规范一致
- 2) 技术安全需求分配给硬件单元的完整性
- 3) 与相关软件安全需求的一致性
- 4) 正确性和精确性

7、硬件设计

这一条款的第一个目标是根据系统设计规范和硬件安全需求设计硬件,第二个目标是验证设计。 硬件设计包括硬件架构设计和硬件详细设计,硬件架构设计应表示出所有硬件单元及彼此间的关系,硬件详细设计是指在电路原理图上的设计。

7.4.1 硬件架构设计

硬件架构应实现硬件的安全要求,每个硬件单元应根据硬件安全要求实现最高的 ASIL。硬件安全要求和实现之间的可追溯性应保存到的硬件单元的最低层,但不需要到硬件详细设计,ASIL 不会分配到硬件元件。

为了避免高复杂性产生的故障,硬件体系架构设计应通过使用表 1 列出原则来具有以下特征:

- a) 模块化
- b) 粒度适当
- c) 简易性

模块化硬件设计属性

Table 1 — Properties of modular hardware design

Properties		ASIL			
		A	B	C	D
1	Hierarchical design 分层设计 精确定于与安全相关的硬件组件	+	+	+	+
2	Precisely defined interfaces of safety-related hardware components	++	++	++	++
3	Avoidance of unnecessary complexity of interfaces 避免不必要的复杂接口	+	+	+	+
4	Avoidance of unnecessary complexity of hardware components 避免不必要的复杂硬件组件	+	+	+	+
5	Maintainability (service) 可维护性	+	+	++	++
6	Testability ^a 可测试性	+	+	++	++

^a Testability includes testability during development and operation.

对于安全相关的硬件组件故障,在硬件设计过程中的非功能性条款应考虑以下的影响:温度,振动,湿度,灰尘,电磁干扰,无论是从硬件结构的硬件组件或从其他它的环境的串扰源。

7.4.2 硬件详细设计

1、为避免设计缺陷，相关的经验教训应该遵循组织的安全文化 2-5.4.7

2、与安全相关的硬件部分失效时应考虑硬件详细设计过程中的非功能性原因，包括以下几方面的影响，如：温度，振动，湿度，灰尘，电磁干扰，噪声系数，无论是从硬件组件的其他部件或它的环境的串扰源。

3、硬件部分的操作条件应满足他们的环境和操作限制的规范。

4、应该考虑稳健设计原则，稳健设计原理，可以利用基于 QM 方法清单。例如保守的组件规范

7.4.3 安全分析

在硬件设计上找出故障原因和故障影响的安全性分析依据表 2 和 ISO 26262-9:2011 条款 8。安全分析的最初目的是支持的硬件设计规范。随后，安全分析可用于硬件设计验证（见 7.4.4）。

硬件设计安全分析
Table 2 — Hardware design safety analysis

Methods		ASIL			
		A	B	C	D
1	Deductive analysis ^a 推理分析	○	+	++	++
2	Inductive analysis ^b 归纳分析	++	++	++	++
NOTE The level of detail of the analysis is commensurate with the level of detail of the design. Both methods can, in certain cases, be carried out at different levels of detail.					
^a A typical deductive analysis method is FTA. 典型的分析方法是 FTA					
^b A typical inductive analysis method is FMEA. 典型的分析方法是 FMEA					

本要求适用于安全目标 ASIL (B)，C，D。每一个安全相关的硬件部件或零件，在确定的安全目标下，安全分析应考虑以下因素：

- A) 安全故障；
- B) 单点故障或残留故障；
- C) 多点故障（或感知，检测或潜在的）。

在大多数情况下，可以将分析限于双点故障。但多点故障比双点故障点故障两个以上的可以显示更高的技术安全概念（例如当实现冗余安全机制）。

双点故障的识别目的是不需要对每一个可能的两个硬件组合的故障进行系统分析，但是，至少，从安全技术概念考虑到组合，（例如两个故障达到或维持一个安全的状态，一个故障影响安全相关的元素，另一故障影响相应的安全机制）。

单点故障

单点故障，是在一个单元中，未被安全机制覆盖且直接会导致违反安全目标的硬件故障。这项规定应用于安全目标 ASIL (B)，C 和 D 的，避免单点故障的有效性安全机制证据，应当提供：

- A) 应提供保持安全状态的安全机制，或安全地切换到安全状态的能力，（特别是恰当的缓解故障的容错时间间隔内的能力）；
- B) 应评估关于残余故障的诊断覆盖率

注意 1：如果诊断测试间隔，加上相关安全机制的故障响应时间，超过有关容错的时间

间隔，可以发生在任何时候（例如，不仅在上电时）的故障不能被认为是有效地覆盖。

注意 2: 如果故障是可以描述为仅发生在上电时刻，车辆行驶过程中，然后接通电源后，对故障执行测试。

注意 3: 采用诸如 FMEA 或 FTA 的分析方法用来组成基本原理。

注意 4: 依据硬件组件和他们相关层的失效模式的知识，评价可以是硬件组件的任一个全局范围的诊断，或更详细的故障模式覆盖的评价。

潜在故障

潜在故障，是在多点故障检测时间间隔内不能被安全机制检测出来的也不能被驾驶员识别的多点故障。 这项规定应用于安全目标 ASIL（B），C 和 D 的，避免潜在故障的有效性安全机制证据，应当提供：

A) 故障检测，并通知到驾驶员的能力，对潜在故障可接受的多点故障检测的时间间隔内，应以确定哪些故障潜伏，哪些故障是不能潜伏的。

B) 对潜在故障的诊断覆盖率进行评价。

注意 1 故障不能被认为是覆盖，如果它的诊断测试间隔，加上相关的安全机构的故障响应 时间，比潜在故障相关的多点故障检测时间间隔长，则认为故障不能被覆盖。

注意 2 采用诸如 FMEA 或 FTA 的分析方法用来组成基本原理。

7.4.4 硬件设计验证

硬件设计应按照 ISO26262-8，第 9 条，针对硬件安全要求合理性和完整性进行验证。为了实现这一目标，在表 3 中列出的方法，应考虑。

硬件设计验证					
Table 3 — Hardware design verification					
Methods		ASIL			
		A	B	C	D
1a	Hardware design walk-through ^a 硬件设计走读	++	++	o	o
1b	Hardware design inspection ^a 硬件设计检查	+	+	++	++
2	Safety analyses 安全分析	In accordance with 7.4.3			
3a	Simulation ^b 仿真	o	+	+	+
3b	Development by hardware prototyping ^b 硬件原型开发	o	+	+	+
NOTE The scope of this verification review is technical correctness of the hardware design.					
^a Methods 1a and 1b serve as a check of the complete and correct implementation of the hardware safety requirements in the hardware design.					
^b Methods 3a and 3b serve as a check of particular points of the hardware design (e.g. as a fault injection technique) for which analytical methods 1 and 2 are not considered to be sufficient.					

注意 此验证审查的范围是硬件设计的技术正确性。

a 方法 1a 和 1b 作为在硬件设计过程中硬件安全要求的完整和正确实施的检查 b 方法 3a 和 3b 作为硬件设计特别点的检查（例如，引入一个故障注入技术），因为分析方法 1 和 2 被认为是不够的。

在硬件设计中，如果发现何硬件安全要求的实施是不可行的，应当按照 ISO26262-8 的变更管理流程发出变更请求。

7.4.5 生产、运行、服务和关闭

如果安全分析已经表明，它们是与安全有关的特殊特性相关的，那么这些特殊特性应被指定。特殊特性的属性应包括：

- a) 生产运行的核查措施
- b) 这些措施的验收标准

8、硬件体系指标评估

这一条款的目的是评估由故障处理的指标要求来设计的项目硬件架构。

这一条款描述了两个硬件体系结构的评价指标的有效性，项目的架构来应对随机硬件故障。这些标准和相关的目标值适用于整个项目的硬件。这些指标涉及的仅限于一些项目的安全相关的电气和电子硬件部分随机硬件故障，即那些能够显著有助于违规或实现的安全目标，对于单点，剩余的和这些部件的潜在故障。对于电气硬件单元，只考虑电气故障模式和故障率。硬件架构指标可以反复的在硬件架构设计和硬件详细设计过程中应用。硬件体系结构指标依赖于产品的整个硬件。符合规定的硬件架构度量的目标值满足项目所涉及的每个安全目标。

定义这些硬件架构指标来实现以下目标：

- 客观上应是可评价的：指标是可验证的，精确区别不同的架构；
- 支持最终设计评估 --使 ASIL 依赖于通过/失败标准
- 揭示安全机制的覆盖面，防止硬件架构单点或残留故障风险；
- 揭示安全机制的覆盖面，防止硬件架构潜在故障风险；
- 提出单点故障、残余故障和潜在故障 --确保硬件故障率不确定性的鲁棒性；
- 限于安全相关的元素
- 支持不同元素的语法，例如目标值能分配给供应商元素

8.4 要求和建议

这项规定安全机制的安全相关的诊断覆盖率应就剩余的故障和有关潜在故障评估 适用于安全目标 ASIL (B)，C 和 D。分析硬件部分的故障率应由以下几个方法确定：

a、使用从工业界认可的工业数据源中得到的硬件故障率数据， 例如，公认的业内人士以确定硬件部分的故障率和故障模式分布，包括 IEC/ TR62380，IEC61709，MIL HDBK217 F 通知 2，RIAC HDBK217 此外，UTE C80-811，NPRD95，EN50129:2003，附件 C，IEC62061:2005，附件 D，RIAC FMD97 和 MIL HDBK338。

b、使用基于现场返回或测试数据。在这种情况下，失效率必须有足够的置信水平，

c、使用工程上基于定量和定性参数的专家判断方法。专家判断应按照结构化的标准来行使这一判断的依据。这些标准应设置故障率的估算 适用于安全目标 ASIL (B)，C 和 D。如果不能提供一个单点故障或潜在故障的计算故障率的充分证据，应提出替代手段（例如添加安全机制来检测和控制这个故障）。

对于每一个安全目标，量化指标值根据 ISO26262-4:2011，7.4.4.2 规定的“单点故障指标”一样，应根据以下参考目标值来源之一：

- A、对硬件架构的指标计算
- B、依据表 4

单点故障指标值

Table 4 — Possible source for the derivation of the target "single-point fault metric" value

	ASIL B	ASIL C	ASIL D
Single-point fault metric	≥90 %	≥97 %	≥99 %

对于每一个安全目标，量化指标值根据 ISO26262-4:2011，7.4.4.2 规定的“潜在故障指标”一样，应根据以下参考目标值来源之一：

- A、对硬件架构的指标计算
- B、依据表 5

Table 5 — Possible source for the derivation of the target "latent-fault metric" value

	ASIL B	ASIL C	ASIL D
Latent-fault metric	≥60 %	≥80 %	≥90 %

9、随机硬件故障对安全目标影响评价

对于随机硬件失效率指标的规定：

Table 6 — Possible source for the derivation of the random hardware failure target values

ASIL	Random hardware failure target values
D	$<10^{-8} \text{ h}^{-1}$
C	$<10^{-7} \text{ h}^{-1}$
B	$<10^{-7} \text{ h}^{-1}$

NOTE The quantitative target values described in this table can be tailored as specified in 4.1 to fit specific uses of the item (e.g. if the item is able to violate the safety goal for durations longer than the typical use of a passenger car).

10、硬件集成和测试

这一条款的目的是通过测试确保开发的硬件满足硬件安全要求。本节所述活动的目的是集成硬件单元和测试硬件设计以验证其符合适当的 ASIL 硬件安全要求。

硬件集成和测试不同于 ISO26262-8:2011，第 13 条的硬件组件活动的限制，它给出了中级层硬件组件符合 ISO26262 的证据。

10.4.1 硬件集成和测试活动，应当按照 ISO26262-8:2011，第 9 条进行。

10.4.2 硬件集成和测试活动，应符合 ISO26262-4:2011，5.5.5 中给出的项目集成和测试计划。

10.4.3 测试设备应该属于质量监控系统

10.4.4 为了使测试规范适合特定的硬件集成测试，测试用例应使用表 10 中所列的方法进行适当组合得到。

硬件集成测试测试用例生成方法
Table 10 — Methods for deriving test cases for hardware integration testing

Methods		ASIL			
		A	B	C	D
1a	Analysis of requirements 需求分析	++	++	++	++
1b	Analysis of internal and external interfaces 内部和外部接口分析	+	++	++	++
1c	Generation and analysis of equivalence classes ^a 等价类产生和分析	+	+	++	++
1d	Analysis of boundary values ^b 边界值分析	+	+	++	++
1e	Knowledge or experience based error guessing ^c 基于知识和经验的错误预测	++	++	++	++
1f	Analysis of functional dependencies 功能依赖分析	+	+	++	++
1g	Analysis of common limit conditions, sequences and sources of dependent failures 分析常见的限制条件、序列和独立失效源	+	+	++	++
1h	Analysis of environmental conditions and operational use cases 分析环境条件和运行用例	+	++	++	++
1i	Standards if existing ^d 存在的标准	+	+	+	+
1j	Analysis of significant variants ^e 分析重要变量	++	++	++	++
^a In order to derive the necessary test cases efficiently, analysis of similarities can be conducted. ^b For example, values approaching and crossing the boundaries between specified values, and out of range values. ^c "Error guessing tests" can be based on data collected through a lessons learned process, or expert judgment, or both. It can be supported by FMEA. ^d Existing standards include ISO 16750 and ISO 11452. ^e The analysis of significant variants includes worst-case analysis.					

A 为了有效地获得必要的测试用例,可以进行相似性分析

B 例如, 接近值和指定范围的边界值, 超出值

C “错误预测测试是基于通过经验教训的过程,或者专家判断,或两者兼而有之, 收集的数据。它可以由 FMEA 来支持

D 现在的标准包括 ISO16750 和 ISO11452 E 重要变量分析包括最差情况分析

10.4.5 硬件集成和测试活动应验证对硬件的安全要求实现的安全机制的完整性和正确性。测试方法如表 11 所示。

Table 11 — Hardware integration tests to verify the completeness and correctness of the safety mechanisms implementation with respect to the hardware safety requirements

Methods		ASIL			
		A	B	C	D
1	Functional testing ^a 功能测试	++	++	++	++
2	Fault injection testing ^b 故障注入测试	+	+	++	++
3	Electrical testing ^c 电气测试	++	++	++	++
^a Functional testing aims at verifying that the specified characteristics of the item have been achieved. The item is given input data, which adequately characterises the expected normal operation. The outputs are observed and their response is compared with that given by the specification. Anomalies with respect to the specification and indications of an incomplete specification are analysed. ^b Fault injection testing aims at introducing faults in the hardware product and analysing the response. This testing is appropriate whenever a safety mechanism is defined. Model-based fault injection (e.g. fault injection done at the gate-level netlist level) is also applicable, especially when fault injection testing is very difficult to do at the hardware product level. For example, showing the response of safety mechanisms to transient faults inside hardware parts, such as a microcontroller, is very difficult to do with fault insertion at the hardware product level since it would require irradiation tests. ^c Electrical testing aims at verifying compliance with hardware safety requirements within the specified (static and dynamic) voltage range.					

A 功能测试的目的在于保证项目规定的特征已经达到。为项目提供输入数据, 使其表现为预期正常运行状态。观察输出并将响应与规定所给相比较, 应对与规范不符合的异常和指示规范不完整的情况进行分析;

B 故障注入测试旨在硬件产品中引入故障并分析其响应。无论安全机制是否定义，本测试方法都是合适的。

C 电气测试旨在规定电压范围内（静态和动态）验证与硬件安全要求的一致性。

10.4.6 硬件集成和测试活动，应当验证硬件抵抗外部应力的健壮性。如表 12 所示

抵抗外部应力下的验证鲁棒性的硬件集成测试

Table 12 — Hardware integration tests to verify robustness and operation under external stresses

Methods		ASIL			
		A	B	C	D
1a	Environmental testing with basic functional verification ^a 各种环境下的功能测试	++	++	++	++
1b	Expanded functional test ^b 扩展功能测试	o	+	+	++
1c	Statistical test ^c 统计测试	o	o	+	++
1d	Worst case test ^d 最坏情况测试	o	o	o	+
1e	Over limit test ^e 过极限测试	+	+	+	+
1f	Mechanical test ^f 机械试验	++	++	++	++
1g	Accelerated life test ^g 加速寿命试验	+	+	++	++
1h	Mechanical Endurance test ^h 机械耐久性试验	++	++	++	++
1i	EMC and ESD test ⁱ EMC试验	++	++	++	++
1j	Chemical test ^j 化学试验	++	++	++	++

ISO26262-6 软件级产品开发

5、 软件级产品开发启动

这个子阶段的目标是计划和启动软件开发的功能安全活动。软件开发的启动是计划活动，其中软件开发子阶段及其支持过程（见 ISO26262-8 和 ISO26262-9）是根据项目发展的程度和复杂性决定和计划。软件开发子阶段和支持流程是通过确定适当的方法启动，以符合有关规定和各自的 ASIL。方法是指通过指南和工具，对于每个子阶段支持。

5.4 要求和建议

5.4.1 应当规划确定产品开发在软件级别的活动和适当的方法。

5.4.2 产品开发的生命周期在软件层面的制定应当按照 ISO26262-2:2011，6.4.3.4 进行，并根据图 2 中给出的参考模型。

5.4.3 如果开发可配置的软件，应参考附件 C。

5.4.4 对于一个项目软件的软件开发过程，包括生命周期，方法，语言和工具，应当是与整个软件生命周期的所有子阶段一致的，并与系统和硬件开发阶段兼容，使得所需的数据可以被正确地转换。

5.4.5 对于软件开发的每个子阶段，下面的选择，包括为他们的指南，应进行：

A 方法

B 工具

5.4.6 选择一个合适的模型或编程语言时必须考虑的标准是：

- a) 一个明确的定义；
- b) 对嵌入式实时软件和运行时错误处理的支持；
- c) 对模块化，抽象化和结构化结构的支持

5.4.7 为了支持设计和执行的正确性，设计和建模语言，或编程语言，应符合表 1 中所列的主题。例如 MISRA C 和 MISRA AC AGC 作为编程语言 C 的编码规范

Table 1 — Topics to be covered by modelling and coding guidelines

Topics		ASIL			
		A	B	C	D
1a	Enforcement of low complexity ^a 低复杂度执行	++	++	++	++
1b	Use of language subsets ^b 语言子集的使用	++	++	++	++
1c	Enforcement of strong typing ^c 类型的执行	++	++	++	++
1d	Use of defensive implementation techniques 使用防守实现技术	0	+	++	++
1e	Use of established design principles 使用既定的设计原则	+	+	+	++
1f	Use of unambiguous graphical representation	+	++	++	++
1g	Use of style guides 使用设计指南	+	++	++	++
1h	Use of naming conventions 使用约定的命名	++	++	++	++
^a An appropriate compromise of this topic with other methods in this part of ISO 26262 may be required.					
^b The objectives of method 1b are — Exclusion of ambiguously defined language constructs which may be interpreted differently by different modellers, programmers, code generators or compilers. — Exclusion of language constructs which from experience easily lead to mistakes, for example assignments in conditions or identical naming of local and global variables. — Exclusion of language constructs which could result in unhandled run-time errors.					
^c The objective of method 1c is to impose principles of strong typing where these are not inherent in the language.					

6、 软件安全需求规范拟定

这个子阶段的第一个目标是拟定软件安全需求，他们是来自技术安全概念和系统设计规范。第二个目标是细化软硬件接口要求，依据 ISO26262-4:2011，第 7 条。第三个目的是验证该软件的安全要求和硬件的软件接口要求与技术安全概念和系统设计规范一致。

6.4 要求和建议

6.4.1 该软件的安全要求应满足每个基于软件的功能，其故障可能违反相应的软件技术安全要求。

例如，功能故障可能导致违反安全规定可以是：

- 使系统达到或保持安全状态的功能；
- 相关的检测，显示和处理的安全相关的硬件元件故障的功能；
- 相关的检测，通知和缓解在软件本身的故障功能；

注 1：这些包括在操作系统和应用程序特定的自我监测的软件来检测，表示和处理系统故障的应用程序。

——与车载和非车载测试相关的功能；

注 2：车载的测试可以由系统本身或所述车辆的运行前和运行后阶段的车载网络内的其它系统进行。

注 3：非车载测试指在生产或服务中与安全有关的功能或性能测试。

- 软件生产和服务过程中进行修改的功能；
- 有关性能或时间要求严格的操作功能。

6.4.2 软件安全要求规范应来源于技术安全概念和系统设计，符合 ISO26262-4:2011，7.4.1 和 7.4.5，应考虑以下因素：

- a) 安全要求符合 ISO 26262-8:2011，第 6 条的规定和管理；
- b) 指定的系统和硬件配置；
配置参数可以包括增益控制，带通频率和时钟分频器。
- c) 有关软硬件接口规范；
- d) 硬件设计规范的有关要求；
- e) 时序约束
- f) 外部接口；
- g) 车辆、系统或硬件的运行模式对软件有影响

6.4.3 如果 ASIL 分解到软件安全技术需求，应遵守 ISO26262-9:2011，第 5 章

6.4.4 在 ISO26262-4:2011，第 7 条发起的软硬件接口规范，详细说明会降至允许正确的控制和硬件的情况，并应说明硬件和软件之间的每一个与安全有关的依赖关系。

6.4.5 如果其它的安全性的要求，除了在 6.4.1 中指定的通过嵌入式软件实施的那些功能，这些功能将被指定，否则引用他们的规范。

6.4.6 软件安全要求的验证，硬件软件接口的规范细化，应按照 ISO26262-8:2011，第 9 条计划。

6.4.7 细化的软硬件接口规范应由负责本系统的硬件和软件开发人员共同进行验

证

6.4.8 软件安全要求和细化的软硬件接口要求应按照 ISO26262-8:2011 第 6 和第 9 章，进行验证：

- a) 与技术安全要求的合规和一致性；
- b) 符合系统设计；
- c) 与软硬件接口一致。

7、 软件体系设计

这个阶段的第一个目标是设计软件体系结构以实现软件安全需求，第二个目标是校验软件体系结构。

7.4 要求和建议

7.4.1 为了确保软件体系设计获取正确和有效地必需的信息来进行后续的开发活动，软件体系结构设计使用的符号应具有适当的抽象水平，如表 2 中所列的软件体系结构设计说明。

软件架构设计符号

Methods		ASIL			
		A	B	C	D
1a	Informal notations 非正式符号	++	++	+	+
1b	Semi-formal notations 半正式符号	+	++	++	++
1c	Formal notations 正式符号	+	+	+	+

7.4.2 在软件体系开发的过程中，应该考虑下列因素：

- a) 软件架构设计的可验证性；
- b) 配置软件的适用性；
- c) 软件单元的设计和实施的可行性；
- d) 软件集成测试中的软件体系结构的可测试性；
- e) 软件体系结构设计可维护性。

7.4.3 为了避免高复杂性造成的故障，软件体系结构设计应使用表 3 中列出的原则具有以下性质：

- a) 模块化；
- b) 封装性
- c) 简单化

软件体系设计原则
Table 3 — Principles for software architectural design

Methods		ASIL			
		A	B	C	D
1a	Hierarchical structure of software components 软件组件的分层结构	++	++	++	++
1b	Restricted size of software components ^a 软件组件的大小限制	++	++	++	++
1c	Restricted size of interfaces ^a 接口的大小限制	+	+	+	+
1d	High cohesion within each software component ^b 软件组件的高内聚性	+	++	++	++
1e	Restricted coupling between software components ^{a, b, c} 软件组件之间的耦合限制	+	++	++	++
1f	Appropriate scheduling properties 合适的调度属性	++	++	++	++
1g	Restricted use of interrupts ^{a, d} 中断使用限制	+	+	+	++
^a In methods 1b, 1c, 1e and 1g "restricted" means to minimize in balance with other design considerations.					
^b Methods 1d and 1e can, for example, be achieved by separation of concerns which refers to the ability to identify, encapsulate, and manipulate those parts of software that are relevant to a particular concept, goal, task, or purpose.					
^c Method 1e addresses the limitation of the external coupling of software components.					
^d Any interrupts used have to be priority-based.					

a 在方法 1B, 1C, 1E 和 1G “限制”是指，设计时考虑尽量减少。

B 例如，方法 1D 和 1E 可以通过分离识别，封装和操作软件相关的能力实现，指特定概念，目标，任务或目的的那些部分。

c 方法 1E 解决软件组件的外部耦合的限制

d 所使用的任何中断都必须基于优先级的

7.4.4 软件体系结构设计应开发到所有软件单元都被识别的水平。

7.4.5 软件体系结构设计应说明：

a) 软件组件的静态设计方面

注 1 静态设计方面包括：

- 软件结构，包括它的等级层次；
- 数据处理的逻辑顺序；
- 数据类型及其特点；
- 软件组件的外部接口；
- 软件的外部接口；
- 约束条件包括架构的范围和外部依赖

注 2：在基于模型的开发的情况下，模型结构是整体模型活动的一个固有部分

b) 软件组件的动态设计方面

注 1 动态设计方面包括：

- 功能和行为；
- 控制流和流程并发；
- 软件组件之间的数据流；
- 在外部接口的数据流；
- 时间限制。

注 2 为了确定动态行为（例如任务，时间片和中断），不同的操作状态（例如，电，关断，正常运行，校准和诊断）应该被考虑。

注 3：为了描述动态行为（例如，任务，时间片和中断），通信关系和他们的分配的

系统硬件（如 CPU 和沟通渠道）应该被指定。

7.4.6 每一个与安全相关的软件组件应被归类为以下之一：

- a) 新开发的
- b) 修改重复利用
- c) 无修改重复利用

7.4.7 新开发的，或经过修改重复使用的安全相关的软件组件，应该符合 ISO26262

7.4.8 那些没有修改重复使用的安全相关的软件组件，则应该符合 ISO26262-8:2011，第 12 条。

7.4.9 该软件的安全要求应分配给软件组件。因此，每个软件组件，应当制定符合任何分配给它的最高 ASIL 要求。

7.4.10 如果嵌入式软件必须实现不同的 ASILs，或安全相关和非安全相关的软件组件，那么所有的嵌入式软件的应按照最高 ASIL，除非软件组件符合的标准与 ISO26262-9:2011，第 6 条共存。

7.4.11 如果软件分区（见附件 D）用于软件组件之间交流，它应确保以下几点：

- a) 共享资源被使用时避免软件分区干扰；
- b) 软件分区是由专用的硬件特性或等效方法来支持（按照 4.3，此规定适用于 ASIL D）；
- c) 实现软件分区的一部分软件开发符合相同或更高的 ASIL 比分配到的软件分区的要求最高 ASIL 更高；
- d) 在软件集成和测试过程中执行软件分区（依据第 10 条）的验证。

7.4.12 按照 ISO26262-9:2011 第 7 条，相关故障的分析应进行，如果软件安全要求的实现依赖于免受干扰或软件组件之间有足够独立性。

7.4.13 依据 ISO26262-9:2011，第 8 条，在软件架构层面安全分析应进行，以便：

- 识别或确认软件的安全相关部分；
- 支持规范和验证的安全机制的效率。

7.4.14 为了详细说明软件体系结构层次的安全机制，按照 7.4.13 安全分析的结果，如表 4 中所列的错误检测机制应应用。

Table 4 — Mechanisms for error detection at the software architectural level

Methods		ASIL			
		A	B	C	D
1a	Range checks of input and output data 输入输出数据范围检测	++	++	++	++
1b	Plausibility check ^a 真实性检测	+	+	+	++
1c	Detection of data errors ^b 数据错误检测	+	+	+	+
1d	External monitoring facility ^c 外部检测设施	o	+	+	++
1e	Control flow monitoring 控制流监控	o	+	++	++
1f	Diverse software design 多样化软件设计	o	o	+	++
^a Plausibility checks can include using a reference model of the desired behaviour, assertion checks, or comparing signals from different sources.					
^b Types of methods that may be used to detect data errors include error detecting codes and multiple data storage.					
^c An external monitoring facility can be, for example, an ASIC or another software element performing a watchdog function.					

A 真实性检查可以包括使用所需的行为的参考模型，assertion 检查，或比较不同来源的信号。

- b 这些方法可用于检测数据中的错误类型，包括错误检测码和多个数据存储。
- c 外部监视设备可以是，例如，专用集成电路或其它软件元件执行一个看门狗功能。

7.4.15 本节适用于 ASIL (A)，(B)，C 和 D，按照 4.3：在软件架构层面指定必要的软件的安全机制，，在安全分析的基础上根据 7.4.13，如表 5 所列的错误处理机制也应适用。

软件架构层面的错误处理机制

Table 5 — Mechanisms for error handling at the software architectural level

Methods		ASIL			
		A	B	C	D
1a	Static recovery mechanism ^a 静态恢复机制	+	+	+	+
1b	Graceful degradation ^b 故障软化	+	+	++	++
1c	Independent parallel redundancy ^c 独立并联冗余	o	o	+	++
1d	Correcting codes for data 数据纠错码	+	+	+	+
^a Static recovery mechanisms can include the use of recovery blocks, backward recovery, forward recovery and recovery through repetition. 静态恢复机制包括使用恢复块，前向恢复，后向恢复和重复恢复					
^b Graceful degradation at the software level refers to prioritizing functions to minimize the adverse effects of potential failures on functional safety. 软件层的故障软化是指优先减少潜在故障对功能安全的不利影响					
^c Independent parallel redundancy can be realized as dissimilar software in each parallel path 独立的并联冗余，可实现在每个并行路径不同的软件。					

7.4.16 在安全目标下，软件体系设计未覆盖的新识别的危险，应写入危险分析和风险评估报告，按照 ISO26262-8:2011，第 8 条的变更管理要求来进行。

- 7.4.17 嵌入式软件的所需的资源，其中包括：
- a) 执行时间；
- b) 存储空间；例如，RAM，栈堆，ROM，非易失性数据。
- c) 通讯资源。

7.4.18 依据 ISO 26262-8:2011, 第 9 章, 软件体系架构应该被校验，使用下面表 6 列出的方法，应具有以下属性：

- a) 遵守软件安全需求；
- b) 兼容目标硬件；
- c) 遵守设计指南。

软件体系设计校验方法

Table 6 — Methods for the verification of the software architectural design

Methods		ASIL			
		A	B	C	D
1a	Walk-through of the design ^a 设计走读	++	+	o	o
1b	Inspection of the design ^a 设计检查	+	++	++	++
1c	Simulation of dynamic parts of the design ^b 设计动态仿真	+	+	+	++
1d	Prototype generation 原型产生	o	o	+	++
1e	Formal verification 正式校验	o	o	+	+
1f	Control flow analysis ^c 控制流分析	+	+	++	++
1g	Data flow analysis ^c 数据流分析	+	+	++	++
^a In the case of model-based development these methods can be applied to the model.					
^b Method 1c requires the usage of executable models for the dynamic parts of the software architecture.					
^c Control and data flow analysis may be limited to safety-related components and their interfaces.					

- a 在基于模型的开发的情况下，这些方法可以应用到该模型
- b 方法 1C 需要软件体系结构可执行模型的动态用法
- c 控制和数据流分析可以限定为与安全有关的部件和接口

8、 软件单元设计和实现

这个子阶段的第一个目标是规定软件单元按照软件体系设计及相关的软件安全要求。第二个目的是实现所指定的软件单元。第三个目标是静态验证软件单元设计和实现。

根据软件体系结构设计，开发软件单元的详细设计，详细设计将被实现为一个模型或直接为源代码，根据建模或编码准则。在进行软件单元测试阶段之前，详细设计和开发是静态验证。如果代码是手工开发，在源代码级别实施相关的属性是可以实现。如果基于模型的开发与自动代码生成时，这些属性应用到模型，无需应用到源代码。

为了开发一个软件单元设计，实现软件的安全要求，以及所有非安全的要求。因此，在安全相关和非安全相关的要求都在这个子阶段的过程中处理。

软件单元的实现包括源代码的生成和编译成目标代码。

8.4 要求和建议

- 8.4.1 本阶段应符合软件单元安全相关的要求。
- 8.4.2 为确保该软件单元设计允许后续的开发活动获取正确和有效地进行所必需的信息，该软件单元的设计应采用表 7 所列的符号说明。

软件单元设计符号
Table 7 — Notations for software unit design

Methods		ASIL			
		A	B	C	D
1a	Natural language 自然语言	++	++	++	++
1b	Informal notations 非正式符号	++	++	+	+
1c	Semi-formal notations 半正式符号	+	++	++	++
1d	Formal notations 正式符号	+	+	+	+

在基于模型的开发与自动生成代码的情况下，该方法为代表的软件单元设计作为基础的代码生成的模型。

8.4.3 软件单元的规范应描述功能行为和内部设计，以达到必要的细节实施水平。例如，内部的设计可以包括使用寄存器和数据存储的限制。

8.4.4 软件单元源代码级的设计与实现应采用如表 8 所列的设计原理，达到以下属性：

- a) 软件单元的子程序和函数的正确执行顺序基于软件架构设计
- b) 软件单元之间的接口的一致性
- c) 软件单元内部的数据流和控制流的正确性；
- d) 简约化
- e) 可读性和可理解性；
- f) 鲁棒性
- g) 软件修改的适用性
- h) 可测试性

Table 8 — Design principles for software unit design and implementation

Methods 在子程序和函数中仅有一个入口和一个出口点		ASIL			
		A	B	C	D
1a	One entry and one exit point in subprograms and functions ^a	++	++	++	++
1b	No dynamic objects or variables, or else online test during their creation ^{ab}	+	++	++	++
1c	Initialization of variables 变量初始化	++	++	++	++
1d	No multiple use of variable names ^a 没有多次使用的变量名	+	++	++	++
1e	Avoid global variables or else justify their usage ^a 避免全局变量或其他用法	+	+	++	++
1f	Limited use of pointers ^a 限制使用指针	o	+	+	++
1g	No implicit type conversions ^{ab} 没有隐式类型转换	+	++	++	++
1h	No hidden data flow or control flow ^c 没有任何隐藏的数据流或控制流	+	++	++	++
1i	No unconditional jumps ^{abc} 没有无条件跳转	++	++	++	++
1j	No recursions 没有递归	+	+	++	++

^a Methods 1a, 1b, 1d, 1e, 1f, 1g and 1i may not be applicable for graphical modelling notations used in model-based development.
方法1a, 1b, 1d, 1e, 1f, 1g和1i未必适用于基于模型的开发中使用的图形化建模符号。

^b Methods 1g and 1i are not applicable in assembler programming. 方法1g和1i并不适用于汇编语言程序设计。

^c Methods 1h and 1i reduce the potential for modelling data flow and control flow through jumps or global variables.
方法1h和1i减少潜在的通过跳转或全局变量建模数据流和控制流。

对于 C 语言，MISRA C 涵盖了许多表 8 所列的方法。

8.4.5 软件单元的设计与实施应按照 ISO26262-8:2011 第 9 条进行验证，并通过使用表 9 列出的验证方法来证明：

- a) 遵守软硬件接口规范（根据 ISO26262-5:2011, 6.4.10）；
- b) 软件安全要求（按照 7.4.9）分配给软件单元的实施的可追溯性；
- c) 源代码和设计规范的一致性
- d) 源代码与编码指南一致性；
- e) 软件单元实现与目标硬件的兼容性。

软件单元设计实现的验证方法

Table 9 — Methods for the verification of software unit design and implementation

Methods		ASIL			
		A	B	C	D
1a	Walk-through ^a 走读	++	+	o	o
1b	Inspection ^a 检查	+	++	++	++
1c	Semi-formal verification 半正式验证	+	+	++	++
1d	Formal verification 正式验证	o	o	+	+
1e	Control flow analysis ^{bc} 控制流分析	+	+	++	++
1f	Data flow analysis ^{bc} 数据流分析	+	+	++	++
1g	Static code analysis 静态代码分析	+	++	++	++
1h	Semantic code analysis ^d 代码分析	+	+	+	+

^a In the case of model-based software development the software unit specification design and implementation can be verified at the model level. 在基于模型的软件开发的情况下，软件单元规格设计和实现可以在模型级进行验证。

^b Methods 1e and 1f can be applied at the source code level. These methods are applicable both to manual code development and to model-based development. 方法1e和1f可以在源代码级应用。这些方法既适用手动代码开发又适合于模型的开发。

^c Methods 1e and 1f can be part of methods 1d, 1g or 1h. 方法1e和1f可以是方法1d, 1g和1h的一部分。

^d Method 1h is used for mathematical analysis of source code by use of an abstract representation of possible values for the variables. For this it is not necessary to translate and execute the source code.

表 9 列出只有静态验证技术。动态验证技术（如测试技术）是包括在表 10, 11 和 12。

9、 软件单元测试

这个子阶段的目标是证明软件单元实现软件单元的设计规范和不含有不需要的功能。依据软件设计规范建立软件单元设计的测试流程，并依照流程来执行。

- 9.4.1 该条款要求应符合如果软件单元是与安全相关的。
- 9.4.2 软件单元测试必须按照 ISO26262-8:2011 第 9 条计划，规定和执行。
- 9.4.3 在表 10 中列出的软件单元测试方法应适用于验证软件单元的实现：
 - a) 遵守软件单元设计规范
 - b) 遵守软硬件接口规范
 - c) 指定的功能
 - d) 不存在非计划的功能
 - e) 鲁棒性
 - f) 足够的资源支持功能

Table 10 — Methods for software unit testing

Methods		ASIL			
		A	B	C	D
1a	Requirements-based test ^a 基于需求的测试	++	++	++	++
1b	Interface test 接口测试	++	++	++	++
1c	Fault injection test ^b 故障注入测试	+	+	+	++
1d	Resource usage test ^c 资源利用测试	+	+	+	++
1e	Back-to-back comparison test between model and code, if applicable ^d	+	+	++	++
^a If possible, model and code between the back-to-back test. The software requirements at the unit level are the basis for this requirements-based test.					
^b This includes injection of arbitrary faults (e.g. by corrupting values of variables, by introducing code mutations, or by corrupting values of CPU registers).					
^c Some aspects of the resource usage test can only be evaluated properly when the software unit tests are executed on the target hardware or if the emulator for the target processor supports resource usage tests.					
^d This method requires a model that can simulate the functionality of the software units. Here, the model and code are stimulated in the same way and results compared with each other.					

- a 在单元级别的软件测试是基于需求的测试
 - b 这包括注射任意故障（通过变量破坏，例如，通过引入代码突变，或通过损坏 CPU 寄存器的值）。
 - c 当软件单元测试是执行在目标硬件或模拟目标处理器，测试的资源使用情况的某些方面才能正确评估
 - d 此方法需要一个模型，它可以模拟软件单元的功能。这里，模型和代码被运行以同样的方式和结果进行相互比较。
- 9.4.4 为了使适当的测试用例按照 9.4.3 软件单元测试规范，测试用例应采用表 11 中列出的方法得出。

Table 11 — Methods for deriving test cases for software unit testing

Methods		ASIL			
		A	B	C	D
1a	Analysis of requirements 需求分析	++	++	++	++
1b	Generation and analysis of equivalence classes ^a 等价类生成和分析	+	++	++	++
1c	Analysis of boundary values ^b 边界值分析	+	++	++	++
1d	Error guessing ^c 错误预测	+	+	+	+
^a Equivalence classes can be identified based on the division of inputs and outputs, such that a representative test value can be selected for each class.					
^b This method applies to interfaces, values approaching and crossing the boundaries and out of range values.					
^c Error guessing tests can be based on data collected through a "lessons learned" process and expert judgment.					

a 等价类可以基于对输入和输出，选择一个代表性的试验值，实验值识别每个类别的边界。

b 此方法适用于接口，接近值，范围值和超过界限值。

c 错误预测测试是基于通过“经验教训”的过程和专家判断收集到的数据

9.4.5 为了评估测试用例的完整性，并证明没有额外功能，在软件单元级别要求的覆盖范围应确定，结构范围应按照表 12 中列出的指标进行测定。如果实现的结构范围被视为是不够的，那么额外的测试用例应指定或提供理由。

Table 12 — Structural coverage metrics at the software unit level

Methods		ASIL			
		A	B	C	D
1a	Statement coverage 语句覆盖	++	++	+	+
1b	Branch coverage 分支覆盖	+	++	++	++
1c	MC/DC (Modified Condition/Decision Coverage) 修正条件/判定覆盖	+	+	+	++

注 1：所述的结构覆盖可以通过使用适当的软件工具来确定。

注 2：在基于模型的开发的情况下，可以在模型层使用的模型类似结构覆盖度量进行结构覆盖分析。

注 3：如果检测的代码用于确定覆盖度的等级，必须表明该仪器具有不受试验结果影响。这可以通过没有仪表代码的重复测试来完成。

9.4.6 软件单元测试的测试环境应尽可能与目标环境密切对应。如果软件单元测试不在目标环境中进行，源代码和目标代码中的差异，以及测试环境和目标环境之间的差异，应在指定目标环境中额外的随后的测试阶段加以分析。

注 1：在测试环境和目标环境之间的差异可以发生在源代码或目标代码，例如，由于不同的位宽度的数据字与该处理器的地址字。

注 2：根据测试的范围内，应使用适当的软件执行单元的测试环境（如目标处理器，处理器仿真器或开发系统）。

注 3：软件单元测试可在不同的环境中被执行，例如：

——模型在环测试

——软件在环测试

——处理器在环测试

----硬件在环测试

注 4：对于基于模型的开发，软件单元测试，可以在模型级进行，在模型和对象代码之间的背对背比较测试之后。背对背的对比测试用于确保该模型的行为对于测试目标等同于自动生成的代码。

10、 软件集成和测试

这部分的第一个目的是集成软件元素，第二个目的是要证明，软件体系结构设计是由嵌入式软件实现。

10.4 需求和建议

10.4.1 软件集成的计划应说明整合各个软件分层单元到软件组件的步骤，直到嵌入式软件完全集成，并应考虑：

- a) 相关的软件集成的功能依赖关系；
- b) 软件集成和软硬件整合之间的依赖关系。

注：对于基于模型的开发，该软件集成，可以在模型层和随后的自动代码生成集成的模型替换为集成。

10.4.2 软件集成测试应根据 ISO26262-8:2011，第 9 条计划，规定并执行。

10.4.3 在表 13 中列出的软件集成测试方法应应用来证明软件组件和嵌入式软件的表现

- a) 遵守第 7 条的软件架构设计；
- b) 符合 ISO26262-4:2011，第 7 条的软硬件接口规范
- c) 规定的功能
- d) 鲁棒性
- e) 充足的资源来支持功能

Table 13 — Methods for software integration testing

Methods		ASIL			
		A	B	C	D
1a	Requirements-based test ^a 基于需求的测试	++	++	++	++
1b	Interface test 接口测试	++	++	++	++
1c	Fault injection test ^b 故障注入测试	+	+	++	++
1d	Resource usage test ^{cd} 资源使用测试 模型和代码间的背对背比较测试	+	+	+	++
1e	Back-to-back comparison test between model and code, if applicable ^e	+	+	++	++
^a The software requirements at the architectural level are the basis for this requirements-based test.					
^b This includes injection of arbitrary faults in order to test safety mechanisms (e.g. by corrupting software or hardware components).					
^c To ensure the fulfilment of requirements influenced by the hardware architectural design with sufficient tolerance, properties such as average and maximum processor performance, minimum or maximum execution times, storage usage (e.g. RAM for stack and heap, ROM for program and data) and the bandwidth of communication links (e.g. data buses) have to be determined.					
^d Some aspects of the resource usage test can only be evaluated properly when the software integration tests are executed on the target hardware or if the emulator for the target processor supports resource usage tests.					
^e This method requires a model that can simulate the functionality of the software components. Here, the model and code stimulated in the same way and results compared with each other.					

a 在单元级别的软件测试是基于需求的测试

b 这包括注入任意故障（通过变量破坏，例如，通过引入代码突变，或通过损坏 CPU 寄存器的值）。

- c 为确保硬件体系结构设计的足够的容量满足需求，如平均和最大处理器性能，最小或最大执行时间，存储使用情况（例如 RAM 用于堆栈和堆，ROM 中的程序和数据）和带宽性能通信链路（如数据总线）必须确定。
- d 当软件单元测试是执行在目标硬件或模拟目标处理器，测试的资源使用情况的某些方面才能正确评估
- e 此方法需要一个模型，它可以模拟软件单元的功能。这里，模型和代码被运行以同样的方式和结果进行相互比较。

10.4.4 为了使适当的测试用例按照 10.4.3 软件集成测试规范，测试用例应采用表 14 中列出的方法得出。

软件集成测试测试用例生成方法

Table 14 — Methods for deriving test cases for software integration testing

Methods		ASIL			
		A	B	C	D
1a	Analysis of requirements 需求分析	++	++	++	++
1b	Generation and analysis of equivalence classes ^a 等价类产生和分析	+	++	++	++
1c	Analysis of boundary values ^b 边界值分析	+	++	++	++
1d	Error guessing ^c 错误预测	+	+	+	+
^a Equivalence classes can be identified based on the division of inputs and outputs, such that a representative test value can be selected for each class.					
^b This method applies to parameters or variables, values approaching and crossing the boundaries and out of range values.					
^c Error guessing tests can be based on data collected through a "lessons learned" process and expert judgment.					

- a 等价类可以基于对输入和输出，选择一个代表性的试验值，实验值识别每个类别的边界。
- b 此方法适用于变量参数，接近值，范围值和超过界限值。
- c 错误预测测试是基于通过“经验教训”的过程和专家判断收集到的数据

10.4.5 为了评估测试用例的完整性，并证明没有额外功能，在软件集成级别要求的覆盖范围应确定。如果实现的结构范围被视为是不够的，那么额外的测试用例应指定或提供理由。

10.4.6 为了评估测试用例的完整性，并证明没有额外功能，在软件集成级别要求的覆盖范围应确定，结构范围应按照表 15 中列出的指标进行测定。如果实现的结构范围被视为是不够的，那么额外的测试用例应指定或提供理由。

软件体系架构覆盖矩阵

Table 15 — Structural coverage metrics at the software architectural level

Methods		ASIL			
		A	B	C	D
1a	Function coverage ^a 功能覆盖	+	+	++	++
1b	Call coverage ^b 调用覆盖	+	+	++	++
^a Method 1a refers to the percentage of executed software functions. This evidence can be achieved by an appropriate software integration strategy. 方法1a中是指所执行的软件功能的百分比。这方面的证据可以通过适当的软件整合策略来实现。					
^b Method 1b refers to the percentage of executed software function calls. 方法1b是指所执行的软件功能的调用的百分比。					

- 注 1：所述的结构覆盖可以通过使用适当的软件工具来确定。
- 注 2：在基于模型的开发的情况下，可以在模型层使用的模型类似结构覆盖度量进行结构覆盖分析。

10.4.7 它应按照 ISO26262-4:2011，第 11 条，验证嵌入式软件作为产品发布的一部分，包含的所有特定功能，并且不包含其他未指定的功能，如果这些功能不损害遵守软件安全要求。

10.4.8 软件集成测试的测试环境应尽可能与目标环境密切对应。如果软件集成测试不在目标环境中进行，源代码和目标代码中的差异，以及测试环境和目标环境之间的差异，应在指定目标环境中额外的随后的测试阶段加以分析。

注 1：在测试环境和目标环境之间的差异可以发生在源代码或目标代码，例如，由于不同的位宽度的数据字与该处理器的地址字。

注 2：根据测试的范围内，应使用适当的软件执行单元的测试环境（如目标处理器，处理器仿真器或开发系统）。

注 3：软件集成测试可在不同的环境中被执行，例如：

- 模型在环测试
- 软件在环测试
- 处理器在环测试
- 硬件在环测试

11、 软件安全需求验证

本章节的目的是验证嵌入式软件完成软件安全需求。

11.4 需求和建议

11.4.1 软件安全要求的验证应按照 ISO26262-8:2011，第 9 条计划，指定和执行。

11.4.2 为了验证嵌入式软件满足软件安全要求，测试应进行在表 16 中列出的测试环境。

软件安全需求验证测试环境
Table 16 — Test environments for conducting the software safety requirements verification

Methods		ASIL			
		A	B	C	D
1a	Hardware-in-the-loop 硬件在环测试	+	+	++	++
1b	Electronic control unit network environments ^a ECU网络环境	++	++	++	++
1c	Vehicles 车辆	++	++	++	++

^a Examples include test benches partially or fully integrating the electrical systems of a vehicle, "lab-cars" or "mule" vehicles, and "rest of the bus" simulations.

a 例如包括测试台部分或完全集成的车辆的电气系统，实验室汽车或“mule”的车辆，并且模拟“总线休眠”

11.4.3 软件安全需求实施的试验应在目标硬件系统上执行。

11.4.4 软件安全需求验证结果应就进行评估：

- a) 符合预期的结果；
- b) 覆盖的软件安全需求
- c) 通过或失败的标准。

ISO26262-7 生产运行

5、 生产

5.1 目标

本条款的第一个目标是开发和维护生产过程中的被安装在道路车辆上的与安全相关的元素或项目。第二个目的是在生产过程中由相关制造商或人员负责该进程来实现功能安全（车辆制造商，供应商，二级供应商，组织等）。

5.2 概述

5.4 建议和要求

5.4.1 生产计划

5.4.1.1 生产流程应通过项目评估来计划，并通过考虑以下因素：

- a) 生产需求
- b) 存储、运输和硬件单元的处理条件，如：硬件元素的允许存储时间
- c) 经产品发布文档认可的配置，
- d) 以前产品生产计划中积累的经验
- e) 生产过程中的适用性，生产资料，工具和测试设备，与安全有关的特殊特性
- f) 人员能力。

5.4.1.2 生产计划应说明生产步骤，顺序和实现该项目，系统或元件的功能安全要求的方法。它应包括：

- a) 生产工艺流程和说明
- b) 生产工具和方法
- c) 可追溯性实现，如标签
- d) 若可以，适用于硬件开发的硬件单元专用措施的实施应符合 ISO26262-5:2011，

9.4.2.4 中规定。

5.4.1.3 作为生产过程的一部分，流程应被定义，以确保正确的嵌入式软件和相关的校准数据被写入的 ECU。

例 1，使用校验和，从而使装载的可执行文件和配置数据的校验和与用于这个特定的模型和车辆配置的正确的校验和进行比较。

例 2，从写入的 ECU 软件读回的零件号与从材料规定的目标零件号比较，以及从特定车辆回读的已加载的校准数据和从材料规定的特定车辆的校准数据比较。

5.4.1.4 当开发生产控制计划，该控件的描述和项目的标准，系统或元件以及与安全相关的特殊特性应予以考虑。

5.4.1.5 顺序和控制步骤方法应在生产控制计划描述，再加上必要的测试设备，工具和测试标准。

5.4.1.6 合理的可预见的过程故障及其对功能安全的影响应该被识别，并实施适当的措施来解决相关过程失败。

5.4.1.7 系统硬件或软件开发层的安全要求的可生产性在生产计划中规定，并指定专门的人负责开发(见 ISO 26262 - 4, ISO 26262 - 5 和 ISO 26262 - 6)。

5.4.1.8 如果系统或元素在生产过程中需要改变，管理过程中所描述的条款 ISO 26262 - 8:2011, , 应当遵守。

5.4.2 试制批量产品

5.4.2.1 试制过程和控制措施应与目标生产工艺对应。

5.4.2.2 试制过程和目标生产工艺之间的差异应进行分析以确定生产过程的哪个部分可以在试制过程哪个部分在目标生产工艺中，并评估两个过程中重要的部分。

注：如果预生产过程中等于目标的生产过程中，评估的结果根据 ISO26262-2:20116.4.9.4 可以在功能安全评估时使用（如生产过程能力的证明）。

例如

偏差涉及生产速度，生产顺序和方法或控制步骤，以及生产的必要手段，测试设备和工具。

5.4.3 生产

5.4.3.1 生产过程及其控制措施应按计划实施和保持。

注：生产人员适当的培训是该实现的一部分。

5.4.3.2 生产过程中的失败（包括其授权范围内的与安全相关的特殊特性的偏差）和对功能安全潜在影响，应分析，应采取相应的措施，保持功能安全能力应当得到验证。例如，这些措施可以包括执行进一步的控制措施，分类，处理和交换元素。

5.4.3.3 以下关于功能安全的能力应当评估和维护

- a) 生产流程
- b) 生产方法
- c) 工具和测试设备

注 1：过程能力可以由周期性过程审核或执行流程步骤的每个人周期性资格认证来证明。

注 2：过程能力覆盖保持与安全相关的特殊特性的能力。

5.4.3.4 测试设备应当受到监控。

5.4.3.5 执行的控制应按照相关的生产控制计划，控制报告应当包括下列信息：控制日期，控制对象的识别和控制结果。

注 1 对于手动控制，控制对象和控制结果的识别就足够了。

注 2 控制对象的识别可以是车辆识别号码或车辆级的控制措施生产号或零件号或控制组件序列号

注 3 控制结果可以由一个状态（如通过或失败）或一组数据边界条件的评价。

5.4.3.6 在生产中发布的版本，只有经过批准的配置才可以被生产，除非发布的产品文档的偏差是由相应负责人的授权。该生产文件版本过后应该更新授权的偏差。

5.4.3.7 在生产阶段出现的生产过程变更应当遵守条款 5。

6、 运行、服务（保养和维护）和关闭

6.1 目标

这一条款的目的是规定项目，系统或元素的客户信息，维修说明以及拆卸说明，以维护汽车生命周期的功能安全。

6.2 概述

这一条款为开发维修说明和用户信息，包括用户需求手册和规划，执行和监控的维护工

作，考虑到该项目的相关安全特点。

6.4 要求和建议

6.4.1 计划

6.4.1.1 操作，维修和保养过程的计划中应项目的评估，并考虑以下情况：

- 1) 维护和修理的要求；
- 2) 要求应提供给用户的信息，确保车辆的安全运行；
- 3) 警告和降级概念；
- 4) 现场数据收集和分析的措施；
- 5) 储存、运输和处理硬件元素的条件；
- 6) 经批准的生产文档版本配置；
- 7) 涉及到的人员能力

6.4.1.2 维护计划应当描述维护步骤或活动顺序和方法、维护间隔和维护的必要手段和工具。

6.4.1.3 维修计划和维修指令应当描述如下：

- 1) 工作步骤、规程、诊断程序和方法；
- 2) 维护工具和手段；
- 3) 控制步骤的顺序和方法，用于验证安全控制标准的特殊特性；
- 4) 项目、系统或元素配置，包括追溯性措施；
- 5) 项目、系统或元素允许的停止，和必要的修改；
- 6) 允许停止和修改的驾驶信息
- 7) 替换零件条款

6.4.1.4 用户信息，包括用户手册，应当提供相关说明和关于物品的正确使用使用警告，以及以下信息：

- 1) 相关的功能描述(即预期使用、状态信息或用户交互)和操作模式；
- 2) 客户行为的描述确保所需的可控制性的失效显示，表示警告和降级的信号；
- 3) 从客户中的警告和降级概念所表示的故障情况下预期的维修活动的描述；
- 4) 关于已知的危害作用的警告结果与第三方产品；
- 5) 关于安全的创新功能项警告，可能导致司机的误解或误用。

6.4.1.5 分解说明书应说明活动之前被应用措施拆装，并且需要防止拆卸，处理过程的违反的安全目标的分解车辆，物品或其元素。

6.4.1.6 系统硬件或软件级安全需求在运行、维护和分解计划中规定，并指定相关负责人负责开发(见 ISO 26262 - 4, ISO 26262 - 5 和 ISO 26262 - 6)。

6.4.2 运行、维护（维修）和分解

6.4.2.1 现场监控过程，涉及到该项目功能安全事故应按计划实施符合 ISO26262-2:2011，7.4.2.4，以便：

- 1) 提供现场数据，应当分析来检测任何存在的功能性安全问题，如果发现，触发行动，解决这些问题，
- 2) 提供所需的证据证明在使用参数，根据 ISO 26262 - 8:2011 条款 14。

6.4.2.2 项目系统或其元素的维护、维修和拆除,应该被管理和记录按照维修计划,和保养、维修指令。

6.4.2.3 零部件的供应和他们的存储和运输根据 6.4.1.3 必须按计划执行。

6.4.2.4 如果后续生产变更是由操作,现场监测, 维护、修理或分解,按照 ISO 26262 - 8:2011 第八条款变更管理过程,应当遵守。

ISO26262-8 支持过程

5、 分布式开发接口

5.1 目标

这一条款的目的是描述程序,并在分布式开发的项目和元素中分配相关的责任。

5.4 要求和建议

5.4.1 要求应用

5.4.1.1 第 5 条的规定应当按照适用于 ISO26262 开发每一个项目和元素,除了现成的硬件部件,下列情况适用:

- 1) 没有特定的硬件安全要求分配给硬件部分;
- 2) 现成的硬件部分具有基于全球质量标准(例如电子元器件 AEC 设计标准)完善的过程和对目标应用程序覆盖范围的参数。

5.4.1.2 客户-供应商关系的要求应适用于所有客户-供应商

注 1 这包括分包了顶级供应商,分包的分包商等

注 2 内部供应商可以相同的方式管理外部供应商

5.4.2 供应商选择标准

5.4.2.1 供应商选择标准应根据 ISO 26262 包括评估供应商的开发和生产项目的复杂性和 ASIL 等级的能力。

注 供应商选择标准包括:

- ✧ 供应商的质量管理体系的证据
- ✧ 供应商过去的性能和质量;
- ✧ 确认供应商的能力的功能安全作为供应商的投标的一部分
- ✧ 根据 ISO 26262 - 2:2011 6.4.9 以前的安全评估的结果
- ✧ 考虑影响的功能安全的从开发、生产、质量和物流部门的汽车制造商部门

5.4.2.2 从客户到供应商候选人申请的 RFQ 应当包括:

- 1) 符合 ISO 26262 的正式请求,
- 2) 项目或功能规范定义的元素,
- 3) 安全目标,功能安全要求或技术安全要求,包括他各自己已经可用的,ASIL 这取决于供应 商报价

5.4.3 分布式开发的启动和规划

5.4.3.1 客户和供应商应指定一个 DIA (开放源码的流程图软件) 包括下列:

- 1) 任命客户和供应商的安全管理人员
- 2) 依照 ISO 26262 - 2:2011 6.4.5 联合定制的安全生命周期

- 3) 定义由供应商执行的活动和过程
- 4) 交付的信息和工作产品,
- 5) 负责该活动的所有负责人
- 6) 目标值的沟通, 源自于系统级目标, 各相关方确认满足单点故障指标的目标值和潜伏性故障指标的目标值, 根据硬件架构的评估指标和评估安全的目标避免硬件随机故障(见 ISO 26262 - 5),
- 7) 支持过程和工具, 包括接口, 保证客户和供应商之间的兼容性。

5.4.3.2 如果供应商进行危险分析和风险评估, 危险分析和风险评估应提供给客户确认。

5.4.3.3 负责项目开发方应根据 ISO26262-3 建立功能安全概念。功能安全要求应在客户供应商之间一致。

5.4.4 分布式开发的执行

5.4.4.1 供应商应向客户报告每个不符合的风险问题, 比如项目计划、安全计划, 集成和测试计划根据 ISO26262-4 或软件验证计划按照 ISO26262-6, DIA 或其他规定。

5.4.4.2 供应商应向客户报告每个异常发生在开发活动区域或其分包商的责任。

5.4.4.3 供应商应确定是否符合每个安全要求。如果没有, 安全概念应当重新审查, 如果有必要, 修改满足安全要求。

5.4.4.4 每个可能影响项目或计划的活动符合 ISO 26262 的安全性的变更根据条款 8 应向另一方获得支持。

5.4.4.5 双方应该考虑根据 ISO26262-2:2011, 5.4.2.7 在类似的项目中以前的经验发展来派生安全要求满足当前的开发。

5.4.4.6 供应商应向客户的安全经理对安全计划中定义的任务和里程碑取得的进展进行报告。报告的格式和交付日期应在供应商和客户之间的达成一致。

例如, 定期间隔, 或当里程碑指定的计划达到, 客户检查发布供应商编制的质量管理报告。

5.4.4.7 双方(供应商或客户)应达成一致履行符合 ISO26262 - 4 的安全验证。

注: 如果供应商执行集成和验证, 由供应商提供所需的能力和资源非常重要, 因为安全验证需要集成的车辆(见 ISO26262-4)。

5.4.4.8 这个需求适用于 ASIL D 按照 4.3。客户应该执行其他功能安全审计在供应商的允许任何适当的时候。

5.4.5 供应商要求下的功能安全评估

5.4.5.1 按照 4.3 本要求适用于 ASILs (B), C, D。在到达里程碑定义时间点一个或多个功能安全评估应当执行, 这些评估应包括项目开发的每个阶段。对于项目的复杂性和 ASILs 的安全目标, 功能安全评估应在适当的细节层。功能安全评估应依照 ISO26262-2:2011 6.4.9 执行。

5.4.5.2 按照 4.3 本要求适用于 ASIL B。功能安全评估应该进行。

注: 这个可以通过客户, 另一个组织或由供应商本身。

5.4.5.3 按照 4.3 本要求适用于 ASILs C 和 D。功能安全评估根据 ISO26262-2:2011

6.4.9. 应当在供应商的要求前提下由客户,或由客户指定的一个组织或个人。

注:这可以通过供应商自身来做。

5.4.5.4 按照 4.3 本要求适用于 ASILs (B), C 和 D。功能安全评估报告应当在客户和供应商的前提下。

5.4.5.5 按照 4.3 本要求适用于 ASILs (B), C 和 D。每个潜在影响交付供应商的异常识别,应分析和寻求根源解决这些冲突。双方应达成协议对谁执行所需的操作。

5.4.6 生产发布

5.4.6.1 供应商应当提供证据给客户依照 ISO26262-2:2011, 和, ISO26262-7:2011, 条款 5 中满足和维护的过程能力。

5.4.6.2 按照 ISO26262-2:2011 7.4.2.1 客户和供应商之间的供应协议应解决功能安全
的责任并定义双方的安全活动。

5.4.6.3 供应协议应描述双方安全相关的交流、生产监控记录特点。

5.4.6.4 任何一方开始意识到安全的事件时应当根据供货协议及时报告。如果一个安全事件发生时,事件应当进行分析。这种分析应包括类似的项目和类似的事件有关各方的潜在影响。

6、 安全需求规范和管理

6.1 目标

第一个目标是针对它们的属性和特点确保正确的安全要求规范。第二个目标是确保的安全需求在整个安全管理生命周期的一致性。

6.2 概述

安全要求构成所有的需求,旨在实现和确保所需的 ASILs 等级。安全生命周期期间,安全要求详细规定在一个层次结构。在 ISO 26262 中安全要求结构和依赖关系如图 2 所示。安全需求分配各单元之中。

安全需求管理包括管理需求,取得协议的要求,获得承诺实施要求,并保持可追溯性。为了支持安全需求管理,应该使用合适的安全需求管理工具。这一条款包括对安全要求的规范和管理要求(见图 3)。

有关的安全要求在不同层级的具体要求内容在 ISO26262-3, ISO26262-4, ISO26262-5 和 ISO26262-6 中列出。

6.4 要求和建议

6.4.1 安全要求规范

6.4.1.1 为了获取表 6.4.2.4 的安全要求特性,安全要求应该通过一个合适的结合方式来详细说明:

(1) 自然语言

(2) 表 1 中列出的方式

注:对于更高级别的安全要求(如功能性和技术性安全要求)用自然语言比较合适的,而对于较低级别的安全要求(如软件和硬件安全要求)用符号在表 1 中列出的是比较合适的。

安全需求说明
Table 1 — Specifying safety requirements

Methods		ASIL			
		A	B	C	D
1a	Informal notations for requirements specification 非正式符号的需求说明	++	++	+	+
1b	Semi-formal notations for requirements specification 半正式符号需求说明	+	+	++	++
1c	Formal notations for requirements specification 正式符号需求说明	+	+	+	+

6.4.2 安全要求的属性和特性

6.4.2.1 安全要求应明确标识为安全要求。

注：为了符合这个要求，安全要求可以列在一个单独的文件。如果安全要求和其它要求被施用在同一文档中，安全性要求可以明确地通过使用一个特殊的属性识别，如

6.4.2.5 中所述。

6.4.2.2 安全要求应当从 ASIL 等级中派生出来，除非 ASIL 分解是按照 ISO26262-9 应用。

注：由于安全目标是顶级的安全要求，ASILs 开始在安全目标水平（见 ISO26262-1:2011，定义 1.108）。

6.4.2.3 安全要求应被分配到每一个项目或元素。

6.4.2.4 安全要求应具有以下特点：

一）清楚和易于理解，

注 1：要求是清楚的，对现存的有关规定的含义达成共识。

注 2：要求是可以理解的，读者在相邻的抽象级别（即无论是利益相关者或该要求的消费者）理解其含义。

二）分单元，

注：在一个分层等级的安全要求是分配在每个单元的，在这样的方式它们不能考虑的单元被分成一个以上的安全要求。

三）内部一致性，

注：与外部的一致性不同，多重安全规定并不互相矛盾，内部一致性是指每一个的安全需求包含在本身没有任何矛盾。

四）可行性

注：一个要求是可行的，它可以在产品开发的限制内被实现（资源、状态灯）。

五）可验证。

6.4.2.5 安全要求应具有以下属性：

一）安全生命周期中一个独特的识别保持不变，

例如，一个需求的独特识别可以通过各种各样的方式，如加下标，每个实例用“应当”一词，例如“系统应该检查…”，每个句子或连续编号包含“应当”一词，如“在……的情况下，系统应检查…”。

二）状态， 例如，实施安全要求的状态可以是“建议”，“假设”，“接受”或“评论”。

三）ASIL。